Ariadne
3D Dungeon Maker

Customize Event Guide

Ver 1.5.1

Chapter 1

Overview

This document explains customizing events in Ariadne 3D Dungeon Maker.

You can customize events by creating scripts for events. Attach this script to an empty GameObject and create a prefab of this object. You can set this object as an event category object in EventCategoryEditor.

You can also set arguments to your event category, and use these arguments in your scripts.

Chapter 2

Events in Ariadne

You can create events (such as open the door, get treasures) in Ariadne.

This chapter explains the structure of event data.

EventMasterData

EventMasterData is a data class to hold data about events. You can set an EventMasterData to a position on the floor of your dungeon. EventMasterData has multiple EventParts that hold conditions to execute events.

EventParts

EventParts has starting conditions of events. Ariadne checks these conditions in EventParts to decide the event to execute. EventParts has a list of EventProcessData.

EventProcessData

EventProcessData is a unit to process events. This data has an object which has an event script. You can set an event category to EventProcessData. EventCategoryEditor defines categories of events.

EventProcessData also has arguments for the event scripts. You can define these arguments in EventCategoryEditor, and set values for these arguments in EventEditor.

Chapter 3

Customizing Events

This chapter explains how to customize events.

Create scripts

Create scripts to process events. When you create an event script, inherit AriadneEventStrategyBase class, and implement the IAriadneEvent interface.

It is useful to duplicate demo scripts of events.

On the next page, there is an example of event scripts.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

namespace Ariadne
{
    /// <Summary>
    /// Event process of wait.
    /// </Summary>
    public class EventWait : AriadneEventStrategyBase, IAriadneEvent
    {
        [SerializeField]
        string argNameWaitTime = "WaitTime";

        /// <Summary>
        /// Execute method of event process.
        /// </Summary>
        /// <param name="callbackObj">GameObject which receives call.</param>
        /// <param name="eventPos">The position of the event.</param>
        /// <param name="processData">The event process data.</param>
        /// <param name="debugMode">The flag for debug.</param>
        public void ExecuteAriadneEvent(GameObject callbackObj, Vector2Int eventPos,
EventProcessData processData, bool debugMode)
        {
            PreEventProcess(callbackObj, debugMode);

            string argValue = processData.eventArgs.Find(arg => arg.argName ==
argNameWaitTime).argListValue[0];
            float waitTime = 0f;
            bool success = float.TryParse(argValue, out waitTime);
            if (!success)
            {
                Debug.LogWarning("Event Position : " + eventPos + " / Argument : " +
argNameWaitTime + " has invalid data. Check your event data in EventEditor.");
            }

            StartCoroutine(WaitProcess(waitTime));
        }

        IEnumerator WaitProcess(float waitTime)
        {
            yield return new WaitForSeconds(waitTime);
            PostEventProcess();
        }
    }
}
```

At runtime, the EventProcessor script instantiates the object which has this event script and executes **ExecuteAriadneEvent()** method in the script.

In this method, execute **PreEventProcess()** method which is defined parent class to set a callback object and set a debug mode flag.

After executing PreEventProcess() method, get arguments of this event. This sample code uses the Find() method in List class and uses the argument name to get data. This code defines the retrieve key as a field and adds **[SerializeField]** attribute to edit this name in the Inspector window.

These argument fields are defined as the list field. If you checked the “Is List” field at EventCategoryEditor, this field has multiple elements. If you unchecked the “Is List” field, this field has an element.

If you defined the argument as a value type, Ariadne holds the data as string type. After getting data of arguments, cast these data to the type you want to use in the script. This sample code casts the argument value to float type.

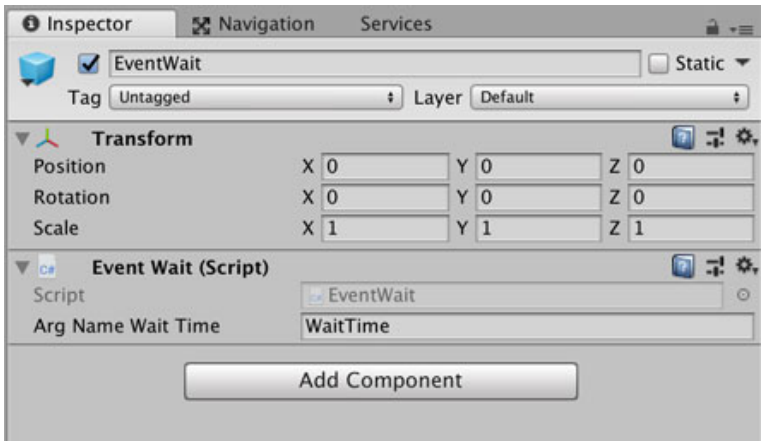
If you defined the argument as a reference type, Ariadne holds the reference for the specified object.

After finishing processes in the script, call **PostEventProcess()**. This method notifies the message of finishing the process to the callback object (generally EventProcessor).

Attach the script

After creating an event script, attach it to an empty GameObject and make it to the prefab file.

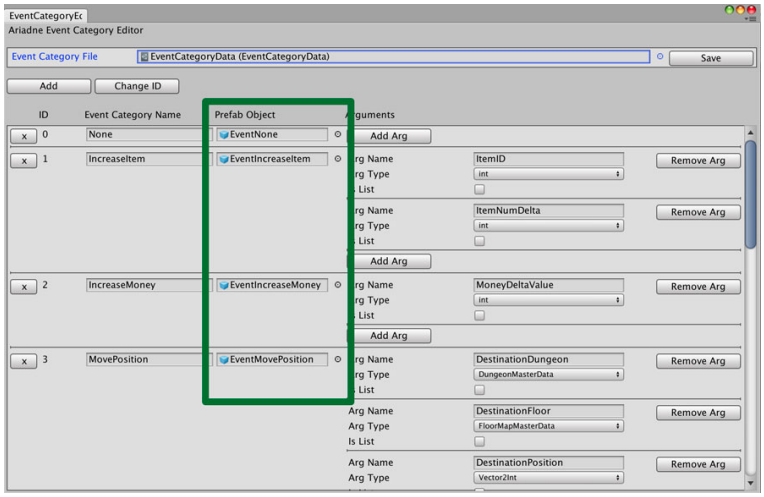
For example, the **"EventWait"** script is attached to the following object.



You can place this prefab in any folder you like. Ariadne places demo prefabs at **[Ariadne/Prefabs/EventObjects]**.

Settings in EventCategoryEditor

Set the prefab object which has an event script to **“Prefab Object”** field in EventCategoryEditor.



You can set arguments for your event script in EventCategoryEditor. These argument name should be matched to the name which you want to use in the script.

See also the manual file to refer EventCategoryEditor in detail.